AI MANTHAN 2.0

Software Requirements Specification

Problem Statement 16

Multimodal Live Conversation Assistant for People with Speech Impairments

Developer-ready specification + API reference + implementation steps + Antigravity guide

Version 1.0 | 31 October 2026
Prepared from the supplied AI Manthan 2.0 PS16 brief

Document status Engineering baseline for a hackathon prototype. Endpoint names and database models are proposed application contracts, not endpoints mandated by the event.

Document Control

Field	Value
Project	PS16 - Multimodal Live Conversation Assistant for People with Speech Impairments
Document	Software Requirements Specification and Antigravity Implementation Guide
Version / Date	1.0 / 31 October 2026
Intended readers	Frontend and backend developers, AI/ML engineer, QA, deployment owner, Antigravity coding agent
Source baseline	Uploaded AI Manthan 2.0 Problem Statement 16, page 1
Implementation stack	Next.js + TypeScript, Python + FastAPI, MongoDB Atlas, ElevenLabs, MediaPipe, LLM
Status	Proposed developer baseline; verify account-specific API capabilities and final stack decisions before implementation

Revision History

Version	Date	Change
1.0	31 Oct 2026	Initial developer-ready SRS, proposed endpoint contracts, workflow and Antigravity guide.

How to read this document

- REQUIRED (PS16): explicitly stated in the uploaded problem statement and must be addressed by the prototype.
- PROPOSED (engineering): recommended design decisions, API paths, schemas, deployment or quality practices added to make the brief implementable.
- OPTIONAL: enhancements that are not necessary to satisfy the core PS16 workflow.

Important distinction The hackathon brief requires the capabilities, not the exact technology choices, route names, database collections or hosting vendors. The API catalogue below is a proposed contract for your team to implement.

Table of Contents

- 1. Executive Summary and Scope
- 2. Source Requirements and Constraints
- 3. Users, Roles and Use Cases
- 4. Functional Requirements
- 5. UI/UX Requirements
- 6. System Architecture and Workflow
- 7. API Specification and Endpoint Reference
- 8. WebSocket Event Contract
- 9. MongoDB Data Model
- 10. AI/ML and Multimodal Processing
- 11. Non-Functional Requirements
- 12. Security, Privacy and Consent
- 13. Testing and Acceptance Criteria
- 14. Implementation Steps and Milestones
- 15. Deployment and Operations
- 16. Antigravity Coding-Agent Guide
- 17. Risks, Assumptions and Open Decisions
- 18. References

1. Executive Summary and Scope

The system is an assistive web application that helps a person with a speech impairment participate in a live conversation. It combines speech recognition with camera-derived gestures, facial landmarks and lip movement, uses conversational context to rank possible intended messages, and speaks a user-confirmed reply in a natural voice.

The interaction must preserve user agency: the AI proposes; the person confirms, edits, rejects or cancels. The system must not silently infer and speak an unconfirmed message.

1.1 Product objective

- Capture/transcribe available speech from the conversation and maintain speaker/context information.
- Analyze relevant hand, face and lip landmarks from a camera stream.
- Fuse audio transcript and visual evidence over time to rank candidate meanings.
- Offer understandable, low-effort reply choices and a clarification path when evidence is ambiguous.
- Generate speech only after the user confirms the reply.
- Support English, Hindi and code-mixed speech, subject to model performance validation.

1.2 In scope - REQUIRED

- ElevenLabs speech recognition and voice generation integration.
- Multimodal analysis using voice together with hand gestures, facial expressions and lip movements.
- Conversation partner transcription/context and suggested replies for the patient.
- LLM-based intent ranking using conversational context.
- English, Hindi and code-mixed language workflows.
- Consent controls for any voice cloning; Voice Design or a standard voice when no recording is available.
- Evaluation of WER, intent accuracy, end-to-end latency (target under 2 seconds) and user confirmation effort.

1.3 Out of scope for initial prototype

- Clinical diagnosis, treatment recommendations or claims of clinical effectiveness.
- Autonomous communication without user approval.
- Guaranteed lip-reading or guaranteed emotion/intent inference from facial expression.
- Training a large speech or multimodal foundation model from scratch.
- Persistent storage of raw camera video or voice recordings by default.
- Native mobile apps unless the team explicitly adds them.

1.4 Success definition

A demonstrable end-to-end path works: partner speaks -> transcript appears -> camera landmarks are captured -> evidence is fused -> ranked replies are shown -> user confirms/edits -> ElevenLabs speaks the confirmed text. The team reports measured performance and limitations rather than claiming unmeasured accuracy.

2. Source Requirements and Constraints

The uploaded one-page PS16 brief identifies people with speech impairments associated with ALS, cerebral palsy, Parkinson’s disease, stroke or laryngectomy as the intended user context. It states that conventional voice-only recognition can fail and that gestures, facial expressions and lip movements should be included.

ID	Requirement from PS16	Classification
SRC-01	Analyze patient voice together with gestures, facial expressions and lip movements.	REQUIRED
SRC-02	Listen to the other person in the conversation.	REQUIRED
SRC-03	Fuse evidence to infer intended message and suggest replies.	REQUIRED
SRC-04	Speak the confirmed message in a natural, personalized voice.	REQUIRED
SRC-05	Use ElevenLabs for speech recognition and voice generation.	REQUIRED

ID	Requirement from PS16	Classification
SRC-06	Support English, Hindi and code-mixed speech.	REQUIRED
SRC-07	Enforce consent for any voice cloning.	REQUIRED
SRC-08	Use ElevenLabs Scribe as the speech-to-text starting point.	PS16 hint
SRC-09	Use MediaPipe or comparable tooling for hand, face and lip landmarks.	PS16 hint
SRC-10	Use an LLM to rank candidate intents with conversation context.	PS16 hint
SRC-11	Use Voice Design if no recordings exist; consider Flash or v3 voice models.	PS16 hint
SRC-12	Try TORGO or UA-Speech, or clearly label simulated atypical speech/gesture data.	PS16 hint
SRC-13	Evaluate WER, intent accuracy, end-to-end latency (target <2s), and user confirmation effort.	REQUIRED evaluation

2.1 Constraints and interpretation

- The under-2-second target is a target to measure, not a guaranteed service-level commitment. Define timer start/end and report median and p95.
- A dataset suggested for speech does not automatically contain synchronized gesture/video labels. Do not imply that TORGO or UA-Speech alone trains a complete multimodal model.
- MediaPipe landmarks are geometric observations. Landmark detection alone does not prove the semantic meaning of a gesture or lip movement.
- Language support must be tested on code-mixed examples; do not infer coverage solely from a vendor's broad language count.

3. Users, Roles and Use Cases

Role	Purpose	Permissions
Primary user / communicator	Uses camera, microphone, phrase buttons and reply suggestions.	Start/end own session; confirm/edit/reject candidates; control voice and accessibility settings; revoke consent; delete own data.
Conversation partner	Speaks naturally to the primary user.	No account required for prototype; may be represented as speaker context. Must be informed that audio is being processed.
Administrator / evaluator (optional)	Runs demo tests and reviews aggregate metrics.	Access test/evaluation data only; no access to private user content unless explicitly authorized.

3.1 Core use cases

Use case	Primary flow	Expected result
UC-01 Start conversation	User grants required browser permissions and starts a session.	Session created; microphone/camera states visible.
UC-02 Hear partner	Partner speaks; app streams/captures audio for Scribe.	Partial/final transcript and available speaker/timing metadata appear.
UC-03 Analyze visual signals	Camera frames are processed by MediaPipe.	Timestamped hand/face/lip features or explicit "not available" state.
UC-04 Infer intended message	Backend aligns transcript and visual features, then asks LLM for ranked possibilities.	Candidate list plus uncertainty/clarification status.
UC-05 Confirm reply	User selects, edits, confirms, rejects or cancels.	Confirmed text is immutable for that generation request.
UC-06 Speak reply	Backend generates audio for confirmed text.	Audio is returned for playback; failures preserve text.
UC-07 End/delete session	User ends session and requests deletion where applicable.	Session closes; retention/deletion policy is applied.

4. Functional Requirements

ID	Feature	Requirement	Acceptance / verification
FR-001	Session lifecycle	System shall create, retrieve and end a conversation session owned by the current user/session token.	Session state changes are persisted and unauthorized users cannot access it.
FR-002	Microphone capture	System shall request microphone permission and capture audio only after explicit user action.	Permission denied/device unavailable has a clear fallback and no silent recording.
FR-003	Partner transcription	System shall transcribe conversation partner speech using ElevenLabs Scribe/Scribe Realtime.	Partial and committed text are distinguished; errors and reconnects are

ID	Feature	Requirement	Acceptance / verification
			handled.
FR-004	Patient speech evidence	System shall process available patient speech and preserve it as a separate evidence stream when technically distinguishable.	The UI does not mislabel speaker identity when diarization is uncertain.
FR-005	Visual landmark extraction	System shall derive hand, face and lip-related landmarks from permitted camera frames.	Timestamped feature objects and missing-detection states are available.
FR-006	Visual feature interpretation	System shall map supported/validated patterns to gesture or movement features.	Unknown or unsupported patterns are marked unknown, not guessed.
FR-007	Temporal alignment	System shall align transcript segments and visual feature windows using timestamps.	Fusion records include source timestamps and alignment window.
FR-008	Multimodal fusion	System shall combine transcript, visual evidence and dialogue context for intent inference.	Both modalities are included when available; missing modality is explicitly recorded.
FR-009	LLM candidate ranking	System shall return a bounded list of plausible candidate replies with rationale and uncertainty fields.	Response schema validates; prompt injection from transcript is treated as untrusted user content.
FR-010	Clarification	System shall ask for clarification or present “not sure” state if confidence is insufficient or modalities conflict.	No unsupported message is auto-selected or spoken.
FR-011	Reply confirmation	System shall support confirm, edit, reject and cancel actions.	Only a confirmed, validated message may be sent to TTS.
FR-012	Speech generation	System shall use ElevenLabs to generate voice output for confirmed text.	Generation is linked to confirmation ID and voice selection; failures retain text.
FR-013	Voice consent	System shall verify active consent before use of a cloned voice.	Revoked/missing consent blocks that cloned voice; permitted standard voice fallback remains.
FR-014	Language selection	System shall expose English, Hindi and Hinglish/code-mixed modes.	Language preference is stored and passed to compatible services where supported.
FR-015	Quick phrases	System may support user-managed saved phrases as an assistive shortcut.	Optional feature; user can create, edit and delete phrases.
FR-016	Session termination/deletion	System shall close a session and apply declared data-retention/deletion rules.	Deletion outcome is auditable without retaining deleted content.
FR-017	Evaluation telemetry	System shall record privacy-minimized timing and evaluation metadata.	WER/intent/latency/effort can be calculated on a labeled test set.

5. UI/UX Requirements

5.1 Main screens

Screen	Required components	Important states
Home / onboarding	Product purpose, Start Conversation, brief permission explanation.	Initial, permission required, permission denied, supported browser check.
Setup	Language (English/Hindi/Hinglish), voice option, accessibility controls, consent explanation.	Loading, saved, unavailable language/model, consent pending/revoked.
Live Conversation	Conversation transcript, speaker labels, camera preview/visual status, suggested replies, large Speak Reply, edit/cancel/end.	Listening, partial transcript, committed transcript, camera unavailable, model processing, ambiguous, reply confirmed, TTS generating, playback/error.
Quick Phrases (optional)	Editable phrase categories and large selectable phrase tiles.	Empty, add/edit/delete, selected.
History (optional, privacy configurable)	Session list and permitted metadata.	No history, loading, deleted, access denied.
Settings / Privacy	Language, voice, text size, contrast, camera/mic controls, voice consent, data deletion.	Permission off, consent active/revoked, save error.

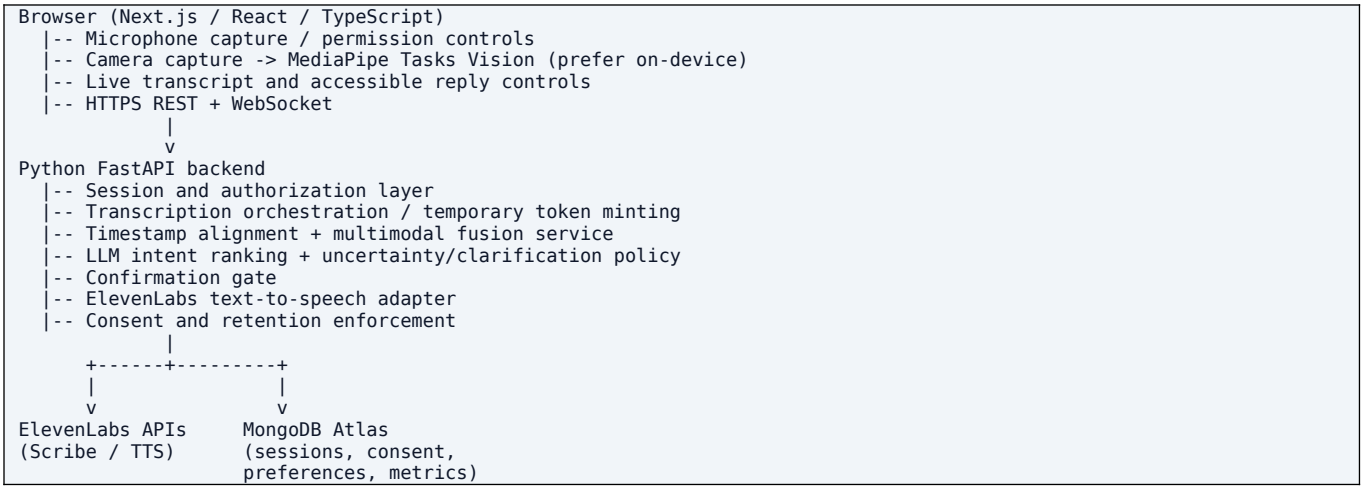
5.2 Accessibility principles

- Large, high-contrast controls with both icon and text label; do not rely on color alone.
- Keyboard operability and visible focus indicators; screen-reader labels for buttons, status and transcript updates.
- Alternative interaction: touch, keyboard, saved phrases and text entry; do not require speech or precise gestures.
- Allow text size and contrast adjustments; keep critical controls in consistent positions.
- Respect reduced-motion preferences; animations must not delay or obscure communication.
- Show microphone/camera active state clearly and provide an immediate stop/end control.

5.3 Visual direction (proposed)

Use a calm, minimal interface: soft off-white background (#F8F9FC), dark navy text (#172554), purple primary action (#6651E8), teal accent (#0F766E), subtle borders and consistent spacing. Suggested controls: 48-56 px minimum button height; use larger touch targets for the main reply actions. These are design recommendations, not PS16 mandates.

6. System Architecture and Workflow



6.1 Component responsibilities

Component	Responsibility	Must not do
Next.js client	Permissions, capture, MediaPipe inference, transcript/reply UI, user confirmation.	Expose long-lived ElevenLabs/LLM/MongoDB secrets; auto-confirm AI guesses.
FastAPI	Auth/session checks, orchestration, schema validation, rate limiting, third-party adapters.	Trust arbitrary client-supplied confidence or consent claims.
MediaPipe	Return hand/face/lip landmark observations with timestamps.	Claim a landmark is a definitive message or emotion.
Fusion service	Normalize, align and package evidence; detect missing/conflicting modalities.	Fabricate visual/audio evidence.
LLM service	Rank plausible candidate messages from structured context.	Execute tool actions or speak without confirmation.
ElevenLabs adapter	Scribe transcription and TTS using server secret or temporary client token.	Expose API key to browser or generate cloned voice without valid consent.
MongoDB	Store minimum required session/configuration/consent/evaluation data.	Store raw audio/video by default.

6.2 End-to-end main flow

1. User opens app, chooses language and explicitly starts a session.
2. Browser checks permissions; camera frames are processed locally by MediaPipe where supported.
3. Audio is streamed to Scribe Realtime or captured in short segments for Scribe batch transcription.
4. Client sends timestamped visual features and committed transcript metadata to FastAPI.
5. Fusion service aligns audio/visual windows and attaches recent dialogue context.
6. LLM returns 2-4 candidate replies or a clarification request; backend validates schema and confidence policy.
7. Frontend presents candidate text with Confirm / Edit / Reject / Not sure.
8. Upon explicit confirmation, backend checks session authorization and consent/voice policy.
9. ElevenLabs TTS generates audio; UI plays it and keeps text visible.
10. Session ends; temporary data is cleared according to retention policy.

7. API Specification and Endpoint Reference

API contract convention All paths below are proposed application endpoints on your FastAPI server (for example https://api.example.com). They are NOT ElevenLabs vendor URLs. Implement them as versioned routes under /api/v1. JSON examples are illustrative contracts.

7.1 Common conventions

- Base path: /api/v1. Content-Type: application/json unless endpoint explicitly uses multipart/form-data or audio bytes.
- Authentication: use an authenticated user token/session for deployed use. For a hackathon demo, a short-lived opaque session token may be used, but it must not be guessable.
- All endpoints must validate session ownership, payload size, allowed MIME types, rate limits and request IDs.
- Timestamps: ISO 8601 UTC. IDs: opaque strings/UUIDs. Never trust client-supplied user_id as authorization.
- Common error envelope: {"error":{"code":"...","message":"...","request_id":"..."}}.

ID	Method	Proposed path	Purpose	Input	Success response	Rules
API-01	POST	/sessions	Create a session	JSON	201: {session_id,status,language,created_at}	Start session and store preferences / policy. No recording starts until explicit permission and capture action.
API-02	GET	/sessions/{session_id}	Get session status	Path	200: {session_id,status,language,started_at}	Owner-only. Return minimal metadata, not raw transcript unless explicitly requested.
API-03	PATCH	/sessions/{session_id}	Update session settings	JSON	200: {session_id,language,voice_mode,updated_at}	Allow only supported settings; validate language: en, hi, hi-en or app-configured codes.
API-04	POST	/sessions/{session_id}/end	End session	Path + optional JSON	200: {status:"ended",ended_at}	Stop capture/streams; mark session ended; run transient-data cleanup.
API-05	DELETE	/sessions/{session_id}	Delete session data	Path	204 No Content	Apply data deletion policy; preserve only legally/operationally required minimal audit marker.
API-06	POST	/transcriptions	Transcribe audio segment	multipart/form-data	200: {transcript_id,text,language,segments[],status}	Batch Scribe adapter. Limit size/duration; validate audio type; do not persist audio by default.
API-07	POST	/realtime/transcription-token	Mint short-lived Scribe token	JSON	200: {token,expires_at,websocket_url}	Backend only. Use vendor-supported single-use token flow where available; never send master API key.
API-08	POST	/visual-features	Submit timestamped landmarks	JSON	202: {feature_id,accepted,received_at}	Accept compact normalized landmarks/gesture observations, not continuous video by default.
API-09	POST	/inference	Run multimodal fusion and candidate ranking	JSON	200: {inference_id,candidates[],needs_clarification,signals_used[]}	Align evidence, create context window, call LLM, validate output; never auto-confirm.
API-10	GET	/inference/{inference_id}	Get inference result	Path	200: {inference_id,status,candidates[],needs_clarification}	Retrieve result for a session owner; candidates are proposals only.
API-11	POST	/sessions/{session_id}/confirmations	Confirm/edit/reject candidate	JSON	200: {confirmation_id,status,confirmed_text?}	Only confirmed/edited message becomes eligible for TTS; reject/cancel never generates speech.
API-12	POST	/speech	Generate confirmed reply audio	JSON	200: audio/mpeg or application/octet-stream	Require valid confirmation ID, session ownership and voice consent if cloned voice is selected.
API-13	GET	/voices	List selectable voices	Query	200: {voices:[{voice_id,label,mode,language_support}]}	Expose only approved/available voice options; do not expose secret or private clone details.
API-14	POST	/voice-consents	Record voice-cloning consent	JSON	201: {consent_id,status,granted_at}	Record purpose, consent version, scope and voice reference; capture consent through clear UI.
API-15	DELETE	/voice-consents/{consent_id}	Revoke voice-cloning consent	Path	204 No Content	Immediately block future cloned-voice generation under that consent.
API-16	GET	/preferences	Get user preferences	Query	200: {language,voice_mode,text_scale,contrast,quick_phrases_enabled}	Return user-owned accessibility and language preferences.
API-17	PATCH	/preferences	Update user preferences	JSON	200: updated preferences	Validate allowed values; never store secrets in preferences.
API-18	GET	/quick-phrases	List saved phrases (optional)	Query	200: {items:[{phrase_id,text,category}]}	Optional enhancement. Owner-only.
API-19	POST	/quick-phrases	Create saved phrase (optional)	JSON	201: {phrase_id,text,category}	Optional enhancement. Validate text length and safe rendering.
API-20	PATCH	/quick-phrases/{phrase_id}	Edit phrase (optional)	JSON	200: {phrase_id,text,category}	Optional enhancement. Owner-only.
API-21	DELETE	/quick-phrases/{phrase_id}	Delete phrase (optional)	Path	204 No Content	Optional enhancement. Owner-only.
API-22	POST	/evaluations/events	Record evaluation event	JSON	202: {event_id,accepted}	Store only minimized metrics; no raw PII unless test protocol explicitly permits.
API-23	GET	/health	Service health	None	200: {status,version,dependencies}	Do not leak keys, internal stack traces or sensitive connection strings.

ID	Method	Proposed path	Purpose	Input	Success response	Rules
API-24	GET	/ready	Readiness check	None	200/503: {ready,dependencies}	Check required dependencies with bounded timeouts; avoid expensive model calls.

7.2 Key request and response examples

API-01 - Create session

```
POST /api/v1/sessions
{
  "language": "hi-en",
  "mode": "live_conversation",
  "accessibility": {"text_scale": "large", "high_contrast": false}
}

201 Created
{
  "session_id": "sess_opaque_id",
  "status": "created",
  "language": "hi-en",
  "created_at": "2026-10-31T08:30:00Z"
}
```

API-06 - Batch transcription

```
POST /api/v1/transcriptions
Content-Type: multipart/form-data
Fields: session_id, audio_file, speaker_hint (optional), language_hint (optional)

200 OK
{
  "transcript_id": "tr_123",
  "text": "Would you like some water?",
  "language": "en",
  "segments": [
    {"start_ms": 0, "end_ms": 1450, "text": "Would you like some water?",
     "speaker": "partner", "is_final": true}
  ],
  "status": "completed"
}
```

API-08 - Submit visual features

```
POST /api/v1/visual-features
{
  "session_id": "sess_opaque_id",
  "captured_at": "2026-10-31T08:30:01.120Z",
  "frame_window_ms": 500,
  "features": {
    "hand_landmarks": [],
    "face_landmarks": [],
    "lip_motion_features": {"available": true, "values": []},
    "gesture_label": "unknown"
  },
  "processing_location": "client"
}

202 Accepted
{"feature_id":"vf_123","accepted":true}
```

Landmark values should be normalized and minimized. In a production implementation, consider sending compact features only when required; do not upload full images/video by default.

API-09 - Multimodal inference

```
POST /api/v1/inference
{
  "session_id": "sess_opaque_id",
  "transcript_ids": ["tr_123"],
  "visual_feature_ids": ["vf_123"],
  "context_window_seconds": 20,
  "language": "hi-en"
```

```
}

200 OK
{
  "inference_id": "inf_123",
  "needs_clarification": false,
  "candidates": [
    {"candidate_id":"cand_1", "text":"Yes, please.",
     "reason":"Consistent with current dialogue and observed input.",
     "confidence":0.72},
    {"candidate_id":"cand_2", "text":"No, thank you.",
     "reason":"Alternative interpretation.", "confidence":0.21}
  ],
  "signals_used": ["transcript", "hand_landmarks", "lip_motion"],
  "created_at":"2026-10-31T08:30:02Z"
}
```

Illustrative values only. LLM confidence is not a calibrated probability unless calibrated against held-out labeled examples. When signals conflict or are missing, the backend should return needs_clarification=true and/or an explicit unknown state.

API-11 - Confirm or edit a reply

```
POST /api/v1/sessions/sess_opaque_id/confirmations
{
  "inference_id": "inf_123",
  "candidate_id": "cand_1",
  "action": "confirm",
  "edited_text": null
}

200 OK
{
  "confirmation_id":"conf_123",
  "status":"confirmed",
  "confirmed_text":"Yes, please."
}
```

API-12 - Generate speech

```
POST /api/v1/speech
{
  "session_id":"sess_opaque_id",
  "confirmation_id":"conf_123",
  "voice_mode":"standard",
  "voice_id":"approved_voice_id",
  "output_format":"mp3"
}

200 OK
Content-Type: audio/mpeg
(binary audio bytes)
```

API-14 - Voice consent

```
POST /api/v1/voice-consents
{
  "voice_id":"user_voice_reference",
  "purpose":"Speak user-confirmed conversation replies",
  "consent_version":"v1",
  "scope":"voice_cloning_and_generation",
  "recording_reference": null
}

201 Created
{"consent_id":"vc_123","status":"active",
 "granted_at":"2026-10-31T08:30:00Z"}
```

8. WebSocket Event Contract

WebSockets are proposed for live transcript updates and low-latency application events. The actual speech stream may connect directly to the vendor using a short-lived token where supported; do not proxy raw audio through application WebSocket unless the chosen architecture requires it.

Channel / event	Direction	Payload	Purpose
/ws/sessions/{id}	Client -> server	{"type":"subscribe","last_event_id":"..."}	Authorize and subscribe to session events.
transcript.partial	Server -> client	{event_id,segment_id,text,language,updated_at}	Interim transcript; may change.
transcript.committed	Server -> client	{event_id,segment_id,text,start_ms,end_ms,speaker}	Finalized segment for inference/context.
vision.status	Client -> server	{captured_at,hand_detected,face_detected,features_available}	Status/feature availability; avoid sending continuous image frames by default.
inference.started	Server -> client	{inference_id,status:"processing"}	Show processing state.
inference.completed	Server -> client	{inference_id,candidates,needs_clarification}	Display candidates for explicit user action.
confirmation.updated	Client -> server	{confirmation_id,action,candidate_id,edited_text}	Record explicit confirmation/edit/reject/cancel.
speech.status	Server -> client	{confirmation_id,status,progress?}	Indicate speech generation and playback readiness.
session.ended	Either direction	{session_id,ended_at}	Stop listeners and clear temporary state.
error	Server -> client	{code,message,request_id,retryable}	Display recoverable error.

8.1 Reconnect rules

- Use a monotonically increasing event ID or cursor to avoid duplicate transcript events.
- Reconnect with bounded exponential backoff; stop retries after session end or user cancellation.
- Treat partial transcripts as replaceable; only committed transcript segments should be used as stable context.
- Never treat a WebSocket connection as authorization by itself; authenticate and verify session ownership.

9. MongoDB Data Model

MongoDB Atlas is the proposed persistence layer. FastAPI is the only application layer that accesses MongoDB. The browser must never receive MongoDB credentials or connect directly to the database.

Collection	Purpose	Key fields (proposed)
users	Account/preferences profile	_id, auth_subject, preferred_language, accessibility, created_at, updated_at
sessions	Conversation metadata	_id, owner_id, status, language, started_at, ended_at, retention_expires_at
conversation_events	Optional minimal transcript/event metadata	_id, session_id, type, speaker, text (only when retention is enabled), start_ms, end_ms, created_at
voice_consents	Consent grant/revocation record	_id, owner_id, voice_ref, purpose, scope, consent_version, status, granted_at, revoked_at
user_phrases	Optional personalized phrases	_id, owner_id, text, category, language, created_at, updated_at
evaluation_results	Test metrics and run metadata	_id, test_run_id, scenario_id, mode, wer, intent_top1, intent_top3, latency_ms, confirmation_actions, created_at
audit_events	Minimal security/consent actions	_id, owner_id, session_id, action, actor, timestamp, request_id

9.1 Sample documents

```
// sessions
{
  _id: ObjectId("..."), owner_id: "user_opaque_id",
  status: "active", language: "hi-en",
  started_at: ISODate("2026-10-31T08:30:00Z"),
  ended_at: null, retention_expires_at: ISODate("...")
}
```

```
// voice_consents
{
  _id: ObjectId("..."), owner_id: "user_opaque_id",
  voice_ref: "vendor_voice_id", purpose: "confirmed_reply_speech",
  scope: "voice_cloning_and_generation", consent_version: "v1",
  status: "active", granted_at: ISODate("..."), revoked_at: null
}

// evaluation_results
{
  test_run_id: "run_001", scenario_id: "hinglish_ambiguous_01",
  mode: "audio_plus_visual", wer: 0.18,
  intent_top1: 0.80, intent_top3: 0.94,
  latency_ms: {p50: 1650, p95: 2380},
  confirmation_actions: 1, created_at: ISODate("...")
}
```

9.2 Indexes and retention

- sessions: unique or query index on session ID; compound index on owner_id + started_at descending.
- conversation_events: session_id + created_at; add TTL only when the chosen retention policy permits automatic expiry.
- voice_consents: owner_id + status; voice_ref where required for consent checks.
- user_phrases: owner_id + updated_at; optional unique owner_id + normalized phrase key.
- evaluation_results: test_run_id + scenario_id + mode.

Privacy default Do not persist raw audio, camera frames, face images or full landmark streams by default. Minimize transcript retention, explain it in the UI, use retention TTL only after policy approval, and verify deletion behavior. A TTL deletion is asynchronous and should not be described as instantaneous.

10. AI/ML and Multimodal Processing

10.1 Speech recognition

- Start with ElevenLabs Scribe batch for short recordings; evaluate Scribe Realtime for the live path.
- Represent partial and committed segments separately. Preserve language and timestamps where returned by the service.
- Speaker diarization/labels are model outputs; mark uncertain identity as unknown instead of assigning a speaker confidently.
- Provide fallback text input and handle unsupported/low-confidence speech without blocking communication.

10.2 MediaPipe vision

- Use Hand Landmarker for hand landmarks and Face Landmarker for facial/lip-related landmarks; choose a documented runtime supported by target browsers/devices.
- Run camera processing on-device where practical. Downsample frames and control inference rate to limit CPU/GPU usage.
- Normalize coordinates and attach capture timestamps. Handle occlusion, poor lighting, camera permission denial and no-face/no-hand states.
- A landmark stream is not a lip-reading system. If semantic lip-reading is attempted, it requires its own labeled evaluation.

10.3 Dataset and training plan

Dataset / data path	Use	Caveat
TORGO	Atypical/dysarthric speech experiments and speech-recognition evaluation.	Does not by itself provide synchronized gestures/facial expression/lip video labels. Check license/access/consent terms.
UA-Speech	Atypical speech robustness evaluation.	Speech data alone is not a multimodal training dataset. Check

Dataset / data path	Use	Caveat
		usage terms.
Consented synchronized capture	Audio + camera/landmark sequence + intended message labels.	Requires participant consent, secure handling and annotation protocol.
Simulated scenarios	Test software integration, event flow and labeled synthetic gesture cases.	Must be labeled simulated; does not establish performance on real speech-impaired users.

Recommended hackathon strategy: establish an audio-only baseline, then compare it with audio + gesture features and audio + gesture + lip-motion features on the same held-out scenarios. Split by participant where possible to reduce person-level leakage. Train a small GRU/1D-CNN gesture sequence classifier only if you have enough labeled data and time; otherwise use transparent feature rules plus LLM context ranking.

10.4 Fusion and ranking contract

```
Input features (illustrative):
- recent partner transcript segments + timestamps
- patient speech transcript (if available)
- gesture labels/features + confidence/availability
- lip-motion temporal features + availability
- language and recent conversation context
- user-specific phrase preferences (only if enabled)

Output:
{
  "candidates": [
    {"text": "...", "reason": "...", "confidence": 0.0}
  ],
  "needs_clarification": true,
  "unknown_or_missing_signals": ["..."],
  "signals_used": ["transcript", "hand_landmarks"]
}
```

- Use strict structured output schema validation; reject invalid candidate objects.
- Treat transcript content as untrusted input, not instructions to the LLM.
- Limit candidates to a small number (e.g., 2-4) and provide an explicit not-sure/clarification path.
- Do not interpret facial expression as a guaranteed emotional state or gesture as universal meaning.
- Do not use uncalibrated LLM confidence as a probability. Calibrate using held-out labeled data if numeric confidence drives thresholds.

11. Non-Functional Requirements

ID	Area	Requirement / target	Verification
NFR-01	Latency	PS16 target: end-to-end latency below 2 seconds; report median and p95.	Instrument timestamps and report defined start/end for representative scenarios.
NFR-02	Availability	Recover gracefully from third-party/API/network failures.	Fault-injection tests; retry and fallback behavior.
NFR-03	Accessibility	Keyboard, screen-reader, large controls, contrast, adjustable text and reduced motion.	Accessibility audit + manual keyboard/screen-reader checks.
NFR-04	Privacy	No raw media storage by default; transparent permission and retention controls.	Inspect storage, logs and deletion workflow.
NFR-05	Security	Secrets server-side; session authorization; payload validation; rate limits.	Secret scan, auth tests, dependency scan and API abuse tests.
NFR-06	Scalability	Bound audio duration, frame rate, request size and concurrent processing.	Load test with stated test environment and workload.
NFR-07	Compatibility	Support current target browsers with camera/mic APIs over HTTPS.	Browser matrix test on demo devices.
NFR-08	Maintainability	Separate adapters for vendor APIs and typed request/response schemas.	Code review and isolated adapter tests.
NFR-09	Observability	Request IDs, stage timings and sanitized errors.	Verify logs exclude secrets and raw media.

12. Security, Privacy and Consent

- Use HTTPS in deployed environments; microphone/camera browser access requires a secure context except local development.
- Keep ELEVENLABS_API_KEY, LLM_API_KEY and MONGODB_URI on the server only. Never place master keys in NEXT_PUBLIC_* variables.
- If client-side Scribe streaming is used, mint a documented single-use/short-lived token on the backend; never expose the account API key.
- Require session ownership checks for all read/write actions; use rate limits and bounded payload size.
- Obtain clear consent before voice cloning. Record consent purpose, scope, version and status; support revocation and block future use after revocation.
- Offer a non-cloned approved voice. A default voice must not be represented as the user's cloned voice.
- Display when microphone/camera are active and provide stop/end controls. Obtain any required consent from conversation partners under applicable rules.
- Avoid raw media persistence and redact transcripts/identifiers from application logs. Encrypt secrets and sensitive data in transit and at rest.
- Provide data deletion controls and document actual deletion/retention behavior, including backups and vendor-side retention.
- Do not position the hackathon prototype as a medical device or clinically validated system.

Vendor data handling Check the selected ElevenLabs account tier and current vendor data-retention controls before processing sensitive recordings. The PS16 brief does not establish HIPAA, medical-device or clinical compliance.

13. Testing and Acceptance Criteria

13.1 Required evaluation metrics

Metric	Definition	Reporting guidance
Word Error Rate	WER = (Substitutions + Deletions + Insertions) / Reference words.	Report by language and speech condition; use human-verified reference transcripts.
Intent top-1 accuracy	Correct intended message in first-ranked candidate / labeled examples.	Report sample count, class balance and held-out split.
Intent top-3 accuracy	Correct intended message appears in top three candidates / labeled examples.	Also report clarification rate and false confident suggestions.
End-to-end latency	Time from defined end of relevant input/utterance to confirmed reply audio readiness/playback.	Report p50/p95, stage breakdown and test conditions; PS16 target <2s.
User confirmation effort	Selections, edits, rejections, clarification turns and time to confirm.	Use scenario-based measure; do not invent user-study outcomes.

13.2 Test cases

Test ID	Scenario	Expected result
TC-01	Microphone permission denied	App explains issue and offers retry/text input; no audio is captured.
TC-02	Partner speech in English	Transcript is displayed with final/partial distinction and speaker uncertainty preserved.
TC-03	Hindi speech	Language mode and transcription flow work; unsupported/uncertain output is disclosed.
TC-04	Hinglish/code-mixed utterance	Transcript and candidate response are evaluated against a labeled code-mixed test case.
TC-05	Hand hidden/occluded	Vision status becomes unavailable; app does not invent a gesture.
TC-06	Face/lip landmarks unavailable	Fusion proceeds with available evidence or asks clarification.
TC-07	Audio and gesture conflict	System asks for clarification; no candidate is automatically spoken.
TC-08	LLM malformed JSON	Backend rejects/repairs safely within policy or returns a recoverable error; UI remains usable.
TC-09	User rejects candidate	No TTS request is issued.
TC-10	User edits then confirms	Only edited confirmed text is sent to TTS.
TC-11	No active voice-clone consent	Cloned voice generation is blocked; standard voice option remains if allowed.
TC-12	Consent revoked	Subsequent generation with that clone is blocked.
TC-13	ElevenLabs unavailable/rate-limited	Text candidate remains visible; error/retry or standard fallback is offered as configured.

Test ID	Scenario	Expected result
TC-14	Session ended	Streams stop, session becomes ended and temporary buffers are cleared.
TC-15	Cross-user session ID access	Request is denied without leaking whether another user session exists.
TC-16	Latency benchmark	All stage timings are recorded; report actual p50/p95 and target status.

13.3 Definition of done

- A live or short-window end-to-end path works without manual backend intervention.
- Audio transcription and camera-derived visual features are both used by inference when available.
- Suggested replies are visible and user-controlled.
- Only confirmed text reaches speech generation.
- English, Hindi and code-mixed scenarios are exercised and limitations are documented.
- Voice-cloning consent and revocation tests pass.
- WER, intent accuracy, latency and confirmation effort are reported from actual tests.
- Deployed demo has secure configuration, clear permission indicators and a rehearsed fallback.

14. Implementation Steps and Milestones

Phase	Suggested time (36h example)	Tasks	Exit criteria
0. Scope and repo setup	0-2h	Inspect repo, freeze core scope, create env files, agree API schemas and Git workflow.	Repo runs; endpoint/data contracts agreed.
1. UI + session skeleton	2-6h	Next.js screens, accessible controls, FastAPI app, session create/end, Mongo connection.	Start/end session works; UI permissions visible.
2. Speech pipeline	6-11h	ElevenLabs Scribe batch/realtime, transcript events, partial/final states.	Partner speech appears as transcript; errors handled.
3. Visual pipeline	11-17h	MediaPipe hand/face landmarks, frame timing, visual status, client-side processing.	Camera produces timestamped features or explicit unavailable state.
4. Fusion + candidate flow	17-23h	Align features, LLM schema, candidate UI, clarification and confirmation gate.	Candidates appear; reject/edit/confirm paths pass.
5. TTS + consent	23-27h	Voice selection, consent store/revoke, ElevenLabs TTS for confirmed message.	Confirmed reply plays; unconfirmed/revoked clone blocked.
6. Evaluation + hardening	27-32h	WER and intent test set, latency instrumentation, access/error tests, privacy review.	Metrics and test results documented.
7. Deploy + demo	32-36h	HTTPS deploy, secrets, smoke test, record fallback, pitch/demo rehearsal.	Public/demo URL works; fallback ready.

14.1 Suggested team split

Workstream	Owner	Deliverables
Frontend/accessibility	Frontend developer	Next.js UI, permissions, transcript, camera status, candidates, confirmations.
Backend/database	Backend developer	FastAPI routes, MongoDB models/indexes, auth/session, consent lifecycle.
Vision/ML	AI/ML developer	MediaPipe landmarks, temporal features, dataset labeling, baseline/fusion evaluation.
AI integration/QA/deployment	Integration owner	ElevenLabs adapters, LLM ranking, tests, latency, hosting and demo story.

15. Deployment and Operations

15.1 Proposed deployment layout

- Frontend: deploy Next.js on Vercel or equivalent HTTPS hosting.
- Backend: deploy FastAPI to a Python-compatible host/container with persistent environment variables and WebSocket support if required.
- Database: MongoDB Atlas with network restrictions, least-privilege database user and backups appropriate to prototype sensitivity.
- AI providers: call ElevenLabs and LLM provider from backend; client-side Realtime speech connection only through short-lived tokens if supported.

15.2 Environment variables

```
# backend/.env (never commit)
MONGODB_URI=mongodb+srv://<user>:<password>@<cluster>/<db>
MONGODB_DATABASE=ps16_assistive_ai
ELEVENLABS_API_KEY=<server_secret>
LLM_API_KEY=<server_secret>
LLM_MODEL=<supported_model_id>
APP_ENV=development
ALLOWED_ORIGINS=http://localhost:3000
SESSION_TOKEN_SECRET=<strong_random_secret>

# frontend/.env.local - public config only
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000/api/v1
```

15.3 Local development steps

11. Install Node.js LTS and Python version selected by the team; pin dependencies.
12. Create MongoDB Atlas database/user and restrict network access.
13. Create ElevenLabs API key and verify the team workspace has access to speech-to-text and text-to-speech features.
14. Create backend virtual environment; install FastAPI, Uvicorn, PyMongo async client, ElevenLabs SDK and validation dependencies.
15. Create Next.js TypeScript app; install MediaPipe Tasks Vision package and UI dependencies.
16. Copy .env.example to local .env files; never commit actual credentials.
17. Run backend health endpoint and frontend dev server.
18. Run a test transcription and test TTS using a standard approved voice before building real-time flow.
19. Test browser camera/microphone permissions over localhost; deploy only on HTTPS.

15.4 Production checklist

- HTTPS configured; CORS allowlist is production frontend origin only.
- Secrets exist only in backend environment/secret manager.
- Database user has least privilege; Atlas network access is restricted.
- Request size/duration/frame-rate and concurrency limits set.
- Rate limiting, timeouts, retry/backoff and graceful service errors implemented.
- Health/readiness checks do not expose secrets or internal stack traces.
- Privacy policy and consent explanation are visible before capture/cloning.
- Deletion, TTL and vendor data handling are tested and documented.
- Smoke tests verify session, transcription, inference, confirmation, TTS and session end.

16. Antigravity Coding-Agent Guide

This section instructs Antigravity to work as a repository-aware implementation agent. The agent must inspect before editing, implement in controlled phases, and report verified status honestly.

16.1 Required repository documents

```
docs/
  SRS.md
  ARCHITECTURE.md
  UI_UX_SPEC.md
  DATABASE_SCHEMA.md
  API_SPEC.md
  AI_PIPELINE.md
  SECURITY_PRIVACY.md
  TESTING_STRATEGY.md
  DEPLOYMENT.md
  IMPLEMENTATION_PLAN.md
AGENTS.md
.env.example
README.md
```

16.2 Antigravity execution protocol

20. DISCOVER: inspect repository tree, package manifests, current routes, database access, environment variable usage, tests and build commands.
21. BASELINE: run existing lint/type-check/test/build; record failures before changing code.
22. MAP: map every SRS requirement and endpoint to existing/new files; identify conflicts, dependencies and missing decisions.
23. PLAN: produce ordered implementation tasks with files to create/edit, acceptance criteria, tests and rollback concerns.
24. IMPLEMENT: work one phase/feature at a time; keep business logic in services, schemas typed and third-party calls behind adapters.
25. VERIFY: run focused tests after each feature; run full lint/type-check/test/build before declaring done.
26. AUDIT: verify endpoint contracts, MongoDB access, consent gate, secret handling, accessibility and data deletion.
27. DOCUMENT: update README/API_SPEC/architecture when code changes contracts.
28. REPORT: classify each requirement as implemented, tested, partially implemented, blocked or not started; provide evidence/commands.

16.3 Master prompt for Antigravity

```
ROLE
Act as a senior software architect and repository-aware full-stack + AI engineer.

PROJECT
AI Manthan 2.0 PS16: Multimodal Live Conversation Assistant for
People with Speech Impairments.

SOURCE OF TRUTH
Read docs/SRS.md, docs/ARCHITECTURE.md, docs/API_SPEC.md,
docs/DATABASE_SCHEMA.md, docs/AI_PIPELINE.md,
docs/SECURITY_PRIVACY.md and docs/TESTING_STRATEGY.md first.
The PS16 brief is the source for mandatory product capabilities.
Do not treat proposed route names as vendor endpoints.

STACK
Next.js + React + TypeScript; Python + FastAPI; MongoDB Atlas;
ElevenLabs Scribe + TTS; MediaPipe Tasks Vision; LLM intent ranking.

NON-NEGOTIABLE RULES
1. Inspect the existing repository before changing files.
2. Do not rewrite working modules without a concrete reason.
3. Do not invent current endpoints, files, dependencies or test results.
4. Keep vendor master API keys and MongoDB URI server-side only.
5. Use schema validation for every external request and LLM response.
6. No TTS generation until explicit user confirmation is verified.
7. Voice cloning requires active consent; revocation blocks future use.
8. Treat missing/ambiguous signals as unknown; do not hallucinate gestures.
9. Prefer on-device MediaPipe processing and avoid raw media persistence by default.
10. Implement requirements in small phases; test each phase.
11. Preserve an accessible text/quick-phrase path if camera/audio fails.
12. Do not claim <2s latency or model accuracy without measurements.

WORKFLOW
A. Analyze repository and report stack, folders, routes, data models,
   existing functionality, environment variables, build/test status.
B. Build a traceability matrix: SRS ID -> code files -> API -> tests.
C. Present implementation plan and risky changes before coding.
D. Implement in this order:
   session/UI skeleton -> MongoDB/session APIs -> Scribe -> MediaPipe ->
   multimodal fusion/LLM -> confirmation gate -> consent/TTS -> evaluation -> deploy.
E. For each phase: modify only required files, add/update tests, run checks,
   summarize changes and unresolved issues.
F. Finish with complete endpoint tests, security/privacy audit,
   accessibility check, production build and deployment guide.

EXPECTED FINAL REPORT
- Requirements traceability with status and evidence
- Files created/modified
- API endpoints implemented and test result
- MongoDB collections/indexes verified
- ElevenLabs and MediaPipe integration status
- Consent and confirmation-gate test results
- WER/intent/latency values with test-set details (or clearly marked pending)
- Deployment URL/status if actually deployed
```

```
- Known limitations and remaining tasks

Begin with repository analysis only. Do not start destructive or broad refactoring.
```

16.4 Requirement traceability template

SRS ID	Code module / file	API	Test ID	Status / evidence
FR-003	backend/app/services/transcription.py	POST /transcriptions; WS transcript events	TC-02, TC-03	Pending implementation
FR-005	frontend/lib/vision.ts	POST /visual-features	TC-05, TC-06	Pending implementation
FR-009	backend/app/services/intent_service.py	POST /inference	TC-07, TC-08	Pending implementation
FR-011	frontend/components/ReplyCandidates.tsx + backend confirmation service	POST /sessions/{id}/confirmations	TC-09, TC-10	Pending implementation
FR-013	backend/app/services/consent_service.py	POST/DELETE /voice-consents	TC-11, TC-12	Pending implementation
NFR-01	backend/app/services/metrics.py	POST /evaluations/events	TC-16	Pending implementation

17. Risks, Assumptions and Open Decisions

Risk / decision	Impact	Mitigation / owner action
Speech impairment variability	Scribe may have high WER for some speakers/conditions.	Test across representative samples; provide text/phrase fallback; disclose limitations.
No synchronized multimodal dataset	Cannot train or fairly evaluate complete fusion model.	Use simulated data for integration only; collect consented synchronized examples for genuine fusion evaluation.
False confident suggestion	Could cause communication error.	Candidate ranking + clarification + user confirmation; never auto-speak.
Latency target miss	Live experience may feel slow.	Measure each stage; reduce frame rate/context size; use realtime STT and low-latency TTS where validated.
Vendor plan/model access	Features, rate limits and credit costs vary.	Verify account access, supported languages/models and pricing before finalizing.
Camera privacy	Sensitive face/gesture data.	On-device landmarks, no raw video upload/storage by default, visible indicator and stop control.
Voice cloning misuse	Identity/consent risk.	Explicit scoped consent, revoke action and standard voice fallback.
Browser/device limitations	Mic/camera or compute unavailable.	Target-device matrix, graceful fallback and demo backup.

17.1 Decisions to confirm before coding

- Which authentication mode is needed for hackathon demo: guest session or account login?
- Will audio stream directly from browser to ElevenLabs using temporary tokens, or pass through FastAPI?
- Which LLM provider/model and structured-output capability will be used?
- Will MediaPipe run in browser or backend? Prefer browser where target devices support it.
- Which speech and gesture datasets are licensed/available for the intended experiments?
- What transcript retention period, if any, is acceptable?
- Which browsers/devices will be used in judging?
- Which languages/code-mixed combinations are included in the labeled test set?

18. References

Primary source: AI Manthan 2.0, Problem Statement 16, “Multimodal Live Conversation Assistant for People with Speech Impairments,” uploaded one-page PDF, page 1. The mandatory requirements and hints in this SRS are based on that source.

Technical references below are official documentation links for implementation verification. Endpoint details and vendor features can change; check the current documentation and your account entitlements during implementation.

- ElevenLabs - Speech-to-text overview - https://elevenlabs.io/docs/overview/capabilities/speech-to-text
- ElevenLabs - Realtime speech-to-text API reference - https://elevenlabs.io/docs/api-reference/speech-to-text/v-1-speech-to-text-realtime

- ElevenLabs - Client-side realtime streaming guide - https://elevenlabs.io/docs/eleven-api/guides/how-to/speech-to-text/realtime/client-side-streaming
- ElevenLabs - API and model overview - https://elevenlabs.io/docs/overview/intro
- MediaPipe Tasks Vision - Web package / Face and Hand Landmarker references - https://github.com/google-ai-edge/mediapipe/tree/master/mediapipe/tasks/web/vision
- MediaPipe - Hand Landmarker guide - https://developers.google.cn/edge/mediapipe/solutions/vision/hand_landmarker
- FastAPI - WebSockets - https://fastapi.tiangolo.com/advanced/websockets/
- MongoDB - PyMongo Async documentation - https://www.mongodb.com/docs/languages/python/pymongo-driver/current/connect/
- Next.js documentation - https://nextjs.org/docs

Appendix A. Recommended FastAPI route layout

```
backend/app/
  main.py
  api/v1/
    sessions.py
    transcriptions.py
    visual_features.py
    inference.py
    confirmations.py
    speech.py
    voices.py
    voice_consents.py
    preferences.py
    quick_phrases.py
    evaluations.py
    health.py
    websocket.py
  core/
    config.py
    security.py
    database.py
    logging.py
  schemas/
    sessions.py
    transcription.py
    vision.py
    inference.py
    speech.py
    consent.py
  services/
    elevenlabs_service.py
    mediapipe_contract.py
    fusion_service.py
    intent_service.py
    consent_service.py
    metrics_service.py
  repositories/
    sessions_repo.py
    consent_repo.py
    events_repo.py
    preferences_repo.py
  tests/
```

Appendix B. Suggested HTTP status codes

Status	Use
200	Successful read/update/action with response body.
201	Resource successfully created.
202	Accepted for asynchronous processing.
204	Successful delete/end where no body is needed.
400	Malformed request or invalid state transition.
401	Missing/invalid authentication.
403	Authenticated but not authorized; consent/ownership restriction.
404	Resource not found or not visible to current owner.
413	Payload/audio segment too large.
415	Unsupported media type.

Status	Use
422	Schema/field validation error.
429	Rate limit or provider quota exceeded.
502/503	Upstream vendor unavailable or service temporarily unavailable.
500	Unexpected server error; never expose internal stack trace.

Appendix C. Demo walkthrough (2-3 minutes)

29. Show onboarding and selected language (English/Hindi/Hinglish).
30. Start session and visibly enable microphone/camera.
31. Conversation partner asks a simple question; show transcript.
32. Show camera/landmark status and one controlled gesture example.
33. Trigger fusion; explain that the assistant ranks possible replies, not mind-reads.
34. Select or edit a candidate and explicitly confirm.
35. Generate and play the confirmed reply using an approved standard or consented voice.
36. Show ambiguous scenario that triggers clarification instead of guessing.
37. Show one evaluation dashboard result and clearly label dataset/simulation source.
38. End session and show privacy/permission controls.

Final implementation principle Build the smallest reliable end-to-end conversation loop first. Add more sophisticated gesture/lip interpretation only when supported by labeled data and measured improvement.